A MAPE-K Instantiation for Dependable Normative Adaptation in Intelligent Agents

Supplementary Material

Proof of Theorem 1. The proof is by induction over the *workflow* structure.

Base case: Let *workflow* be a Task. AdaptWorkflow either returns this task unchanged (Algorithm 1, line 7)—if no SLEEC rules refer to this task, or returns a Sequence to which the task is added in line 12, and whose other tasks are all derived from SLEEC rules. In both scenarios, the resulting workflow is equivalent to the original task (in the sense specified in the theorem).

Inductive hypothesis: Assume that the theorem holds for: (i) the sub-workflows of the Sequence from lines 17–22; (ii) the body of every option of the Decision from lines 23–27; and (iii) the body of the Loop from lines 28–30.

Inductive step: (i) Let *workflow* be a Sequence of sub-workflows. AdaptWorkflow returns a new *sequence* (line 22) that combines the SLEEC-ADAPT*ed* variants of these sub-workflows, preserving their original order. Since the theorem holds for the SLEEC-ADAPT*ed* sub-workflows (by inductive hypothesis), it will also hold for their combination within the returned *sequence*. (ii) Let *workflow* be a Decision. AdaptWorkflow returns a new Decision with the same number of options, each with the same guard as an original option, and the SLEEC-ADAPT*ed* variant of that original option's body. As the theorem holds for these SLEEC-ADAPT*ed* option bodies (by inductive hypothesis), and the guards are unchanged in the returned *sequence*, the theorem will also hold for this *sequence*. (iii) An argument similar to that used for a Decision in (ii) shows that the theorem also holds when *workflow* is a Loop.

Having covered all cases from the switch statement starting in line 2 of AdaptWorkflow (Task in the base case, and Sequence, Decision and Loop in the inductive step), we conclude that the theorem holds for any *workflow*. □

Proof of Theorem 2. AdaptWorkflow performs a depth-first traversal of the input *workflow*, processing each task exactly once (lines 3–16). For every task, it determines the SLEEC rulesets associated with the task's start and end in lines 4 and 5, respectively. SLEEC compliance is achieved by adding tasks derived from each *rule* within these rulesets via the function GenTasksFromSLEEC, both before the original task and after the original task (lines 8–12). We show that this function operates correctly by induction over the number of defeaters d in the *rule*.

Base case: When the *rule* handled by GenTasksFromSLEEC contains no defeaters ($d = 0$), a task corresponding to the *rule*'s response is created in line 32. This task is then placed into the body of a Decision if the *rule* specifies a triggerGuard (lines 33–34). The resulting sub-workflow—which ensures compliance with the SLEEC *rule*—is returned without further modification, as the call to AddDefeaters (line 35) has no effect in the absence of defeaters.

Inductive hypothesis: Assume that for any SLEEC *rule* with fewer than N defeaters ($d < N$), the sub-workflow returned by GenTasksFromSLEEC ensures compliance with that *rule*.

Inductive step: Consider a *rule* with $d = N$ defeaters, where $N > 0$. The resulting sub-workflow is returned following the call to AddDefeaters from line 35. This function handles the sequence of $N > 0$ defeaters recursively (line 40) by:

1. Making a recursive call to itself with a sequence only containing the first $N - 1$ defeaters. By the inductive hypothesis, this call returns a sub-worklow that satisfies the SLEEC rule with respect to those $N - 1$ defeaters.
2. The N-th defeater is then handled by constructing a two-option Decision (line 40). The first of these options performs the task derived from the N-th defeater's response (if that defeater's guard holds), while the second option executes the correct sub-workflow associated with the first $N - 1$ defeaters. This precisely matches the intended semantics of the SLEEC *rule*, ensuring that the sub-workflow returned by GenTasksFromSLEEC complies with the entirety of the *rule*, as required by the theorem. □

Proof of Theorem 3. GetRulesByTrigger (lines 4 and 5) amounts to a 'for' loop inspecting and appending rules to a set. Thus it has time complexity $\Theta(r)$. The auxiliary function AddSubWorkflow appends a sub-workflow to a Sequence by setting the pointer of the last element from the original Sequence to the first element of the appended sub-workflow. This is a constant-time operation.

AdaptWorkflow is divided into four cases. When the *workflow* is a Task (the base case, line 3), two calls are made to GetRulesByTrigger ($\Theta(r)$) to compile relevant SLEEC rulesets. A loop through the first ruleset (*startRules*) follows (lines 9–11) and invokes GenTasksFromSLEEC and AddSubWorkflow to derive sub-workflows from each of the up to r rules from *startRules*. These have time complexities of $\mathcal{O}(d + 1)$ (since GenTasksFromSLEEC calls AddDefeaters once, and this function then invokes itself recursively at most of d times, once for each defeater from the *rule* handled by GenTasksFromSLEEC), and $\Theta(1)$, respectively. This gives $\mathcal{O}(r(d+1))$ complexity for this loop. Another loop of analogous structure and time complexity follows (lines 13–15). Thus, the complexity to handle the *workflow*'s t tasks is $\mathcal{O}(tr(d + 1))$.

For other cases, AdaptWorkflow iterates through the *workflow*'s sub-workflows. For Decisions, Loops and every task of Sequences, a constant-time operation is performed each iteration to add the adapted sub-workflow (excluding the time to adapt individual tasks, whose complexity we already considered for the base case). Thus, the non-base cases of AdaptWorkflow add a time complexity of $\Theta(w)$, where w is the number of workflow elements.

Combining the complexities derived so far results in an overall time complexity of $\Theta(w) + \mathcal{O}(tr(d + 1))$, which simplifies to $\mathcal{O}(tr(d+1))$ as the effect of w is negligible. □